

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I, K.Mounith(192425090)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Mounith

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	
---------------	-----	--	--	-------------------------------	---------------------------	--

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

A. Real-Time Face Recognition System — Edge Deployment

Objective

Design a real-time face recognition system using OpenCV that can run on a low-power device such as a Raspberry Pi. The system should recognize faces from a predefined dataset while working under different lighting conditions and using limited CPU and memory.

End-to-End Computer Vision Pipeline

Live Camera → Image Acquisition → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Decision → Final Output

Stage 1: Image Acquisition

- Capture live video using a camera connected to the Raspberry Pi.
- Use OpenCV's VideoCapture() to read frames.
- Reduce the input resolution, for example to 640 × 480, to reduce CPU usage.
- Process frames continuously from the live video stream.
- Stage 2: Preprocessing

Each captured frame is prepared before face detection.

Steps:

1. Resize the frame to a smaller size.
2. Convert the image from BGR to grayscale.
3. Apply histogram equalization or CLAHE to improve visibility under poor lighting.
4. Optionally apply Gaussian filtering to reduce image noise.

Example:

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
enhanced = clahe.apply(gray)
```

Stage 3: Face Detection

Use a lightweight OpenCV face detector such as:

- Haar Cascade Classifier, or
- OpenCV DNN face detector.

For a Raspberry Pi with limited resources, a lightweight Haar Cascade can be used when maximum efficiency is required.

```
faces = face_cascade.detectMultiScale(
    enhanced,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30,30)
)
```

The detector identifies the location of each face using bounding boxes:

```
[
(x,y,w,h)
]
```

Stage 4: Face Region Extraction and Normalization

For every detected face:

1. Crop the face region.
2. Resize it to a fixed size such as 100×100 pixels.
3. Normalize pixel intensity.
4. Optionally align the face using detected facial landmarks.

```
face = enhanced[y:y+h, x:x+w]
```

```
face = cv2.resize(face, (100, 100))
```

Stage 5: Feature Extraction

Instead of storing or comparing large raw images, extract compact facial features.

For a lightweight system, LBPH (Local Binary Patterns Histograms) can be used with OpenCV.

LBPH describes local texture patterns around facial regions.

Why LBPH?

- Lightweight
- Requires relatively little memory
- Works well for small predefined datasets
- Can be implemented directly using OpenCV
- Suitable for CPU-limited edge devices

Stage 6: Face Recognition

Train the recognizer using the predefined face dataset.

Example dataset:

Dataset/

```
├── Person_1/
│   ├── image1.jpg
│   ├── image2.jpg
│   └── image3.jpg
├── Person_2/
│   ├── image1.jpg
│   ├── image2.jpg
│   └── image3.jpg
```

The LBPH recognizer is trained using these labeled face images.

During live operation:

Detected face → Feature extraction → LBPH comparison → Person ID + confidence

If the similarity/confidence is acceptable:

Person Recognized: Person_1

Otherwise:

Unknown Person

Stage 7: Decision Making

Set a recognition threshold to prevent incorrect identification.

For example:

Confidence within acceptable threshold → Recognized

Confidence outside threshold → Unknown

This is important because the system should not force every detected face to match someone in the predefined dataset.

Stage 8: Final Output

Display the result on the live video:

```
+-----+
|           |
|  [ Face Detected ]  |
|           |
|  Person: Alice      |
|  Confidence: Good   |
|           |
+-----+
```

OpenCV can draw the bounding box and name:

`cv2.rectangle(frame, (x,y), (x+w,y+h), ...)`

`cv2.putText(frame, name, (x,y-10), ...)`

The final output can be:

- Recognized person's name
- Unknown-person message
- Face bounding box
- Optional access-control signal or alert

Complete Pipeline

Live Camera



Frame Acquisition



Resize Frame



Grayscale Conversion



CLAHE / Lighting Correction



Face Detection



Face Cropping



Face Resizing & Normalization



LBPH Feature Extraction



Face Recognition



Confidence / Threshold Check



Known Person / Unknown Person



Display Result on Live Video

Final Justification

The proposed system uses OpenCV with a lightweight face-detection and LBPH recognition pipeline. Reducing the frame size and processing only detected face regions minimizes CPU and memory usage, making it suitable for a Raspberry Pi. Grayscale conversion and CLAHE improve performance under varying lighting conditions. The predefined dataset provides the known identities, while the recognition threshold handles unknown faces. Therefore, the pipeline provides a practical, low-resource, real-time face recognition solution for edge deployment.

2. Automated Video Surveillance with Alert System A

security firm needs a video surveillance system that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

A . Automated Video Surveillance with Alert System

1. Objective

The aim is to develop an OpenCV-based automated surveillance system that monitors CCTV video continuously, detects motion or suspicious activities, and generates an alert in real time.

Main Requirements

Detect moving objects and unusual activities.

Generate alerts immediately.

Work with normal CCTV camera/video feeds.

Reduce false alarms caused by rain, trees, shadows, lighting changes, etc.

Run efficiently in real time.

2. Complete System Pipeline

CCTV Camera → Video Acquisition → Preprocessing → Background Subtraction → Motion Detection → Feature/Contour Detection → Activity Analysis → Decision → Alert → Video/Frame Storage

3. Image/Video Acquisition

The CCTV camera provides a continuous video stream.

OpenCV can read the video using:

```
cap = cv2.VideoCapture(0)
```

For an existing CCTV/video file:

```
cap = cv2.VideoCapture("surveillance.mp4")
```

Purpose

Capture frames continuously.

Provide input to the detection system.

Support standard camera or recorded CCTV footage.

4. Preprocessing

Before detecting motion, each frame is improved.

Steps

Convert frame to grayscale.

Apply Gaussian Blur.

Resize the frame if necessary.

Original Frame



Grayscale



Gaussian Blur



Processed Frame

5. Background Subtraction

Background subtraction separates the moving foreground from the stationary background.

A suitable OpenCV method is MOG2.

```
bg = cv2.createBackgroundSubtractorMOG2()
```

```
fgmask = bg.apply(frame)
```

Example

Background:

Empty road

Current frame:

Person walking

Background Subtraction:

Person = Foreground

Road = Background

Moving trees

Changing illumination

Small background movements

Camera scene changes

6. Motion Detection

The foreground mask contains the detected moving regions.

Contours are extracted from the mask.

```
contours, _ = cv2.findContours(  
    fgmask,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

For each contour, calculate its area.

```
area = cv2.contourArea(contour)
```

Very small regions can be ignored.

For example:

Area < threshold → Ignore

Area ≥ threshold → Possible object

Why?

Small movements may be caused by:

Noise

Leaves

Shadows

Camera vibration

Ignoring very small regions reduces false positives.

7. Feature Detection

After detecting a moving region, the system can extract useful features such as:

Bounding box

Object area

Object position

Movement direction

Movement speed

Movement duration

Example:

Moving Object



Bounding Box



Position + Size + Direction

8. Unusual Activity Detection

Simply detecting motion is not enough because normal movement should not always generate an alert.

The system can define rules.

Example 1: Restricted Area

If a person enters a predefined restricted region:

Person detected



Inside restricted area?



YES

↓
ALERT

Example 2: Long-Duration Motion

If an object remains in a restricted area for a certain amount of time:

Object detected

↓
Track object

↓
Remains for long duration

↓
Suspicious activity

↓
ALERT

Example 3: Sudden Movement

If the object's position changes rapidly:

Frame 1 → Position A

Frame 2 → Position B

Frame 3 → Position C

↓
Large movement

↓
Possible unusual activity

9. Reducing False Positives

False positives occur when the system generates an alert even though there is no real security problem.

For example:

Tree movement

Rain

Shadows

Camera noise

Headlights

Changing sunlight

Methods to reduce false positives

1. Gaussian Blur

Removes small noise.

2. Minimum Contour Area

Ignore very small moving regions.

3. Adaptive Background Subtraction

MOG2 adapts to gradual background changes.

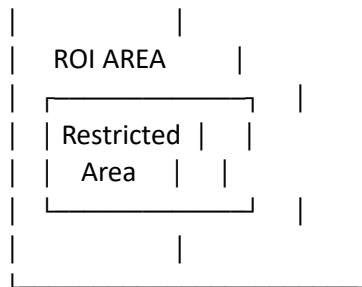
4. Motion Duration

Do not generate an alert for a single changed frame. Require motion to continue for several frames.

5. Region of Interest (ROI)

Monitor only important areas.

Entire CCTV Frame



This prevents irrelevant areas from generating alerts.

10. Alert Generation

When suspicious activity is confirmed, the system generates an alert.

Possible outputs:

Display warning on screen.

Play an alarm sound.

Save the suspicious frame.

Record the suspicious video segment.

Send a notification if connected to an external service.

Example:

⚠ ALERT

Suspicious Activity Detected

Time: 10:35:20

Location: Restricted Area

11. Video Storage

When an alert occurs, the system can save:

Current frame

Timestamp

Short video clip

This allows security personnel to review the event later.

12. Real-Time Processing

The system must process CCTV frames quickly.

Techniques used

Resize video frames.

Use grayscale images.

Use lightweight background subtraction.

Process only the required ROI.

Ignore small contours.

Avoid unnecessary complex calculations.

Pipeline

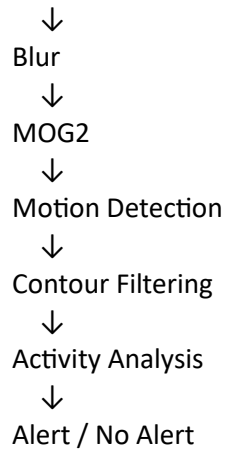
CCTV Frame



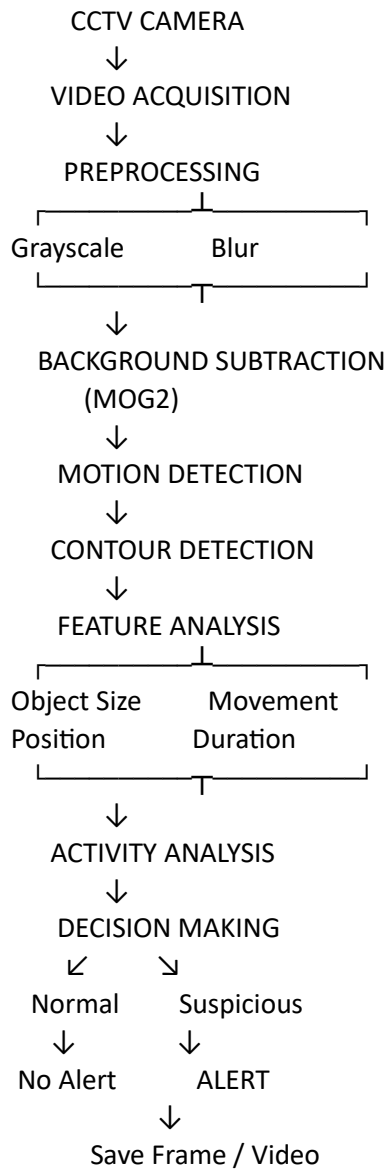
Resize



Grayscale



15. Final System Architecture



3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

OCR System for Low-Quality Documents

Objective

The goal is to build an OCR (Optical Character Recognition) system using OpenCV that can extract text accurately from low-resolution, noisy, and poorly illuminated scanned documents.

Complete Pipeline

Input Document → Preprocessing → Noise Removal → Image Enhancement → Binarization → Segmentation → Morphological Processing → Text Extraction → OCR → Final Text

1. Image Acquisition

Read scanned document images using OpenCV.

Convert the image into a suitable format for processing.

```
img = cv2.imread("document.jpg")
```

Purpose: Provides the input image for the OCR system.

2. Image Resizing

Increase the resolution of low-quality documents.

```
img = cv2.resize(img, None, fx=2, fy=2)
```

Why?

Larger characters make individual letters easier to recognize.

3. Grayscale Conversion

Convert the color image into grayscale.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Why?

OCR mainly needs text intensity information. Grayscale reduces the data from three color channels to one, making processing faster.

4. Noise Removal

Apply Gaussian or median filtering.

```
gray = cv2.GaussianBlur(gray, (3,3), 0)
```

Why?

Removes:

Scanning noise

Small dots

Salt-and-pepper noise

Minor artifacts

This prevents noise from being incorrectly recognized as characters.

5. Illumination Enhancement

Use CLAHE (Contrast Limited Adaptive Histogram Equalization).

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```

```
enhanced = clahe.apply(gray)
```

Why?

CLAHE improves local contrast and helps when different parts of the document have different brightness levels.

6. Binarization / Thresholding

Convert the enhanced grayscale image into black and white.

```
binary = cv2.adaptiveThreshold(  
    enhanced, 255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11, 2  
)
```

Why adaptive thresholding?

It works better when illumination is uneven.

Original Image

↓

Grayscale

↓

Enhanced Image

↓

Black + White Text Image

7. Morphological Processing

Use morphological operations such as opening and closing.

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (2,2))
```

```
clean = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
```

Why?

Removes small unwanted objects.

Connects broken parts of characters.

Improves character shape.

8. Text Segmentation

Separate text from the background and identify:

Text lines

Words

Individual characters

Contours can be used for segmentation.

```
contours, _ = cv2.findContours(  
    clean,
```

```
cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE  
)
```

Why?

Separating text regions makes it easier for the OCR engine to recognize individual characters.

9. OCR Recognition

The processed image is given to an OCR engine such as Tesseract OCR.

Processed Image



OCR Engine



Recognized Characters



Text

Example:

Image:

"COMPUTER VISION"

OCR Output:

"COMPUTER VISION"

Batch Processing

For a digitization company, many documents need to be processed automatically.

Document 1 → Preprocessing → OCR → Text 1

Document 2 → Preprocessing → OCR → Text 2

Document 3 → Preprocessing → OCR → Text 3



Continue



All extracted text saved

The system can process all images from a folder without manual intervention.

Complete Pipeline

Scanned Document



Image Acquisition



Resize / Upscaling

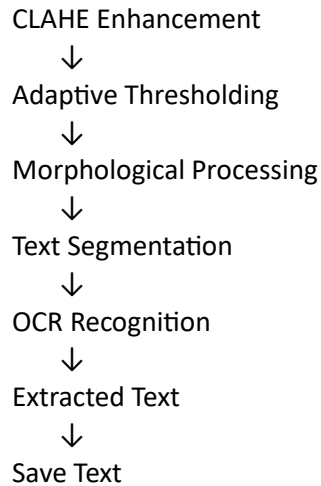


Grayscale Conversion



Noise Removal





4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

A. Facial Expression Recognition for Mobile Application

Objective

Develop a lightweight OpenCV-based system that detects a face and recognizes three expressions:

- 😊 Happy
- 😞 Sad
- 😐 Neutral

The system should work in real time on a mobile CPU.

Complete Pipeline

Camera → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Expression Classification → Output

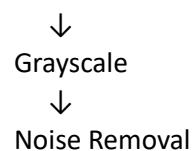
1. Image Acquisition

- Capture live frames using the mobile camera.
- Resize frames to a smaller size such as 320×240 .
- Process selected frames instead of every frame if needed.

2. Preprocessing

- Convert the frame to grayscale.
- Apply Gaussian Blur to reduce noise.
- Use histogram equalization or CLAHE for lighting changes.

Camera Frame





Contrast Enhancement

3. Face Detection

Use an OpenCV Haar Cascade or lightweight OpenCV DNN face detector.

```
faces = face_cascade.detectMultiScale(gray)
```

The detector identifies the face using a bounding box.

4. Face Alignment

- Crop the detected face.
- Resize it to a fixed size.
- Use simple landmark information such as the position of the eyes to compensate for slight head movement.

Detected Face



Eye/Face Position



Alignment



Normalized Face

5. Feature Extraction

Extract important facial features such as:

- Eye position
- Eyebrow movement
- Mouth shape
- Cheek movement

For a lightweight solution, LBP/HOG features can be used instead of a large deep-learning model.

6. Expression Classification

Train a lightweight classifier such as SVM using labeled facial-expression images.

Facial Features



SVM



Happy Sad Neutral

7. Temporal Smoothing

To avoid the expression changing rapidly between frames, use the results from several consecutive frames.

Example:

Frame 1 → Happy

Frame 2 → Happy

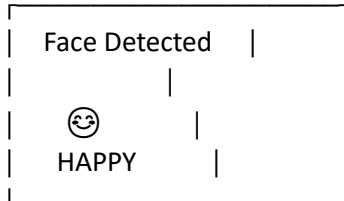
Frame 3 → Happy

Frame 4 → Happy

Final → HAPPY

8. Final Output

Display the detected expression on the mobile screen.



Complete System

Mobile Camera



Frame Resize



Grayscale + CLAHE



Face Detection



Face Cropping & Alignment



Feature Extraction



SVM Classifier



Happy / Sad / Neutral



Temporal Smoothing



Display Expression

5. Smart Traffic Monitoring System A

city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Smart Traffic Monitoring System

Objective

A. Complete Pipeline

Live CCTV → Video Acquisition → Preprocessing → Vehicle Detection → Vehicle Tracking → Counting → Visualization → Statistics

1. Video Acquisition

Capture live CCTV video using OpenCV VideoCapture().

Read frames continuously.

```
cap = cv2.VideoCapture("traffic.mp4")
```

2. Preprocessing

Resize the frame to reduce processing time.

Convert to grayscale if required.

Apply Gaussian Blur to reduce noise.

CCTV Frame



Resize



Noise Removal



Processed Frame

3. Vehicle Detection

Use a lightweight detector such as background subtraction (MOG2) for a fixed CCTV camera.

```
bg = cv2.createBackgroundSubtractorMOG2()
```

```
mask = bg.apply(frame)
```

The moving vehicles are identified from the foreground mask.

For more accurate detection, an OpenCV-compatible YOLO/DNN detector can be used.

4. Vehicle Tracking

After detection, assign an ID to each vehicle and track its position across consecutive frames.

Frame 1 → Car ID 1

Frame 2 → Car ID 1

Frame 3 → Car ID 1

Frame 4 → Car ID 1

The vehicle's center point can be calculated from its bounding box:

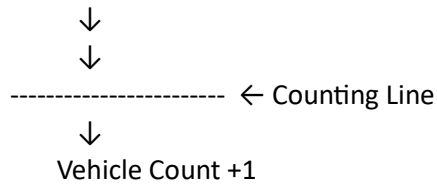
```
[  
Center=( $x+\frac{w}{2}$ , $y+\frac{h}{2}$ )  
]
```

A simple centroid-based tracker can be used for a lightweight system.

5. Vehicle Counting

Create a virtual counting line across the road.

VEHICLE



When the center of a tracked vehicle crosses the line:

```

[
Count = Count + 1
]
  
```

6. Visualization

Use OpenCV drawing functions:

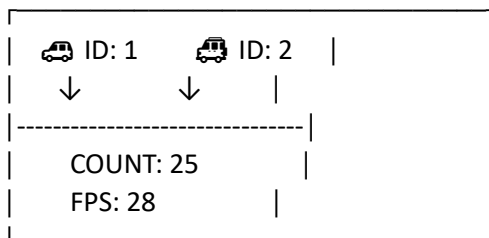
cv2.rectangle() → vehicle bounding box

cv2.putText() → vehicle ID and count

cv2.line() → counting line

cv2.circle() → vehicle center point

Example display:



7. Traffic Statistics

The system can display

Total vehicles

Cars/trucks/buses if classification is used

Vehicles per direction

Current vehicle count

Processing FPS

Example:

Total Vehicles : 125

Cars : 90

Buses : 15

Trucks : 20

FPS : 28

Final System Flow

Live CCTV



Video Capture



Resize + Noise Removal

